

The Next Inflection Point in AI Interaction

How vLLM-Omni Enables Models to
Listen and Speak Simultaneously

全双工，与社区共同构建

致谢 60+ 贡献者 · 160+ 相关 PR 已合入

 Oxtoward	 Oz5a	 AbelSara	 akshatvishu	 amy-why-3459	 andyluo7	 anurag12-webster	 BeatSeat	 BruceLoveDecimal	 CarrotSwordsman	 chickeyton
 cong729	 dshah1333	 eval-dev	 FayeSpica	 frank-2077	 Fyrqo8	 Gaohan123	 GKangaroo	 guanzheh	 herotai214	 Hiro208
 hsluustc0106	 HyperbolicCr	 IneshReddy249	 LHXuuu	 linyueqian	 LinzeShi	 lishunyang12	 LOGO127	 m0g3r	 MrDongsIs	 natureofnature
 NickCao	 NolenLiang	 NumberWan	 psv666	 pujitha24	 R2-Y	 RyanYun09	 sphinxkkkbc	 StormMelody	 stringl11111	 Sy0307
 tc-mb	 TheCodeWrangler	 THUqliu	 Tiagosf00	 tlysanhuo	 twu3202	 vraiti	 vuuihc	 wuli666	 xsmccc	 xuhuan51
	 y-null	 yenuo26	 yuekaizhang	 ZacheryAU	 ZhengWG	 Zhou248	 zyforsure			

代表性已合入 PR

Native duplex 运行时 · #3907	Unified duplex 框架 · #7413	OpenAI Realtime API · #7285	Qwen3-Omni 接入 · #7585
PersonaPlex unified duplex · #7695	Nemotron VoiceChat · #6089	AURA 接入 · #7633	有界输出传输 · #7644

截至 2026 年 10 月 7 日

让音视频会话持续进行

01 输入输出并行

助手回应时，仍持续接收用户语音与画面。

02 流式响应

文本与音频增量输出，无需等待完整回答。

同一段实时会话

摄像头画面



用户语音



助手语音



barge-in

03 实时交互调整

结合新输入，继续回应、调整后续内容或让出话语权。

硬打断：强制停止

客户端 / 服务端控制

显式 cancel 或配置的 VAD 策略

软打断：自主让出话语权 • 依赖模型能力

模型结合持续输入与上下文，自主决定何时停止发言。

不同模型如何处理实时交互

以 Web demo 为例：模型决策、Server VAD 与客户端操作共同决定交互行为。

模型	交互方式	交互效果
MiniCPM-o 4.5	持续接收语音与视频输入 模型决定 LISTEN / SPEAK	在持续会话中聆听或回应 默认不依赖 Server VAD
Qwen3-Omni	Server VAD 检测用户语音 持续接收麦克风输入	清空待播音频 打断当前 response*
AURA	按住说话，松开后 提交新的音视频输入	按住时停止本地播放 服务端仍可继续生成

应用可接入显式 `response.cancel`，取消服务端当前 `response`。

模型让出话语权

模型切换到聆听状态
LISTEN 不等于 `response.cancel`

本地停播

停止用户听到的声音
服务端仍可继续生成

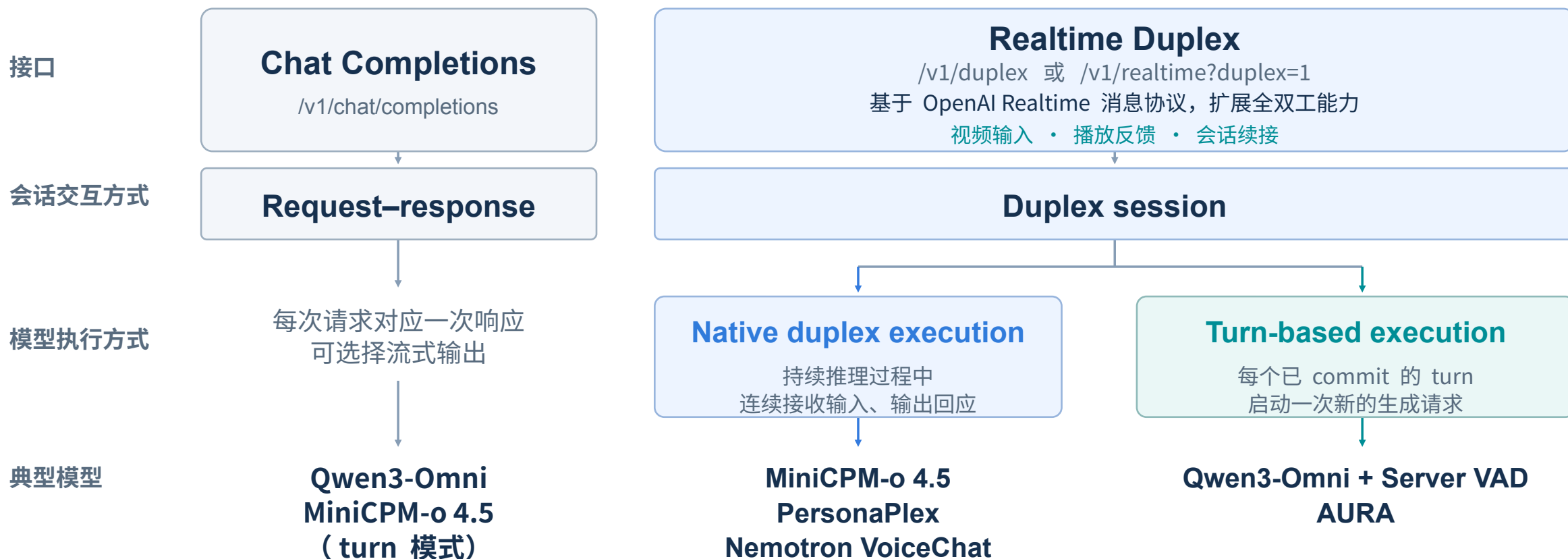
服务端取消

取消当前 `response`
客户端清空待播音频

* Server VAD 打断需启用 `interrupt_response`；具体行为取决于模型与所选 demo profile。

接口、会话交互与模型执行方式

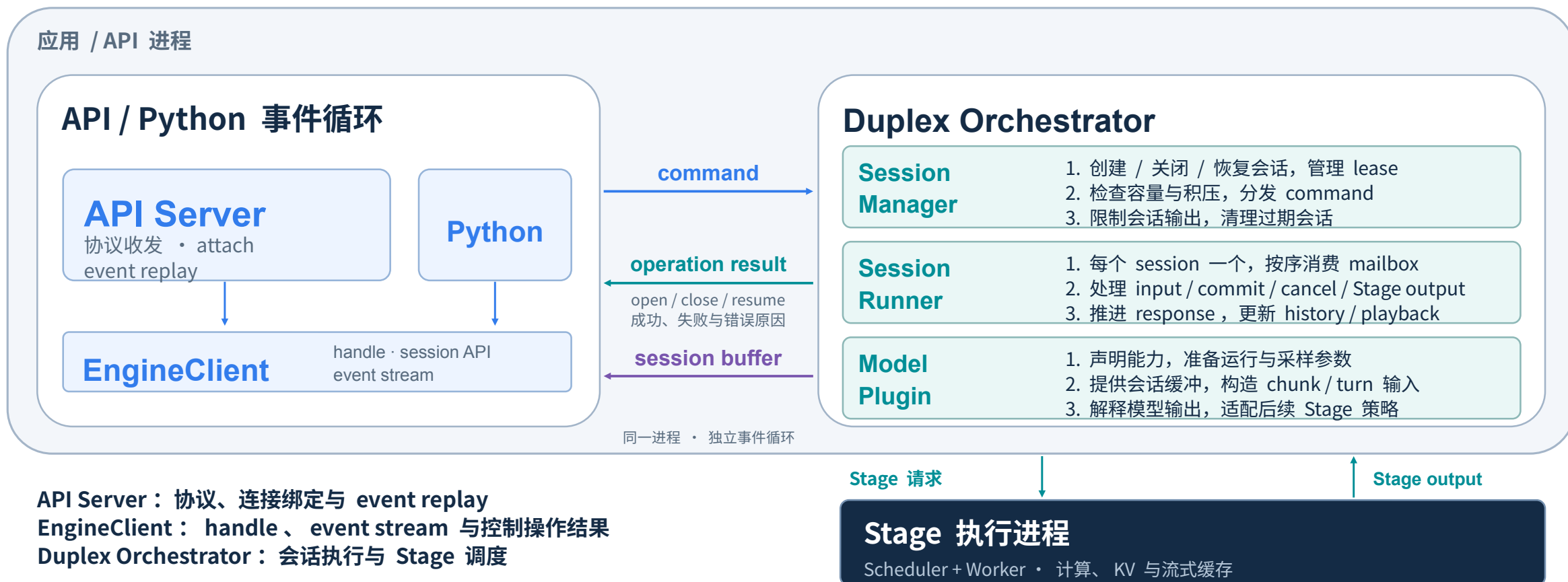
接口承载会话交互；执行方式决定模型如何处理输入。



Realtime 另有独立的 turn-based 路径：/v1/realtime?duplex=0。

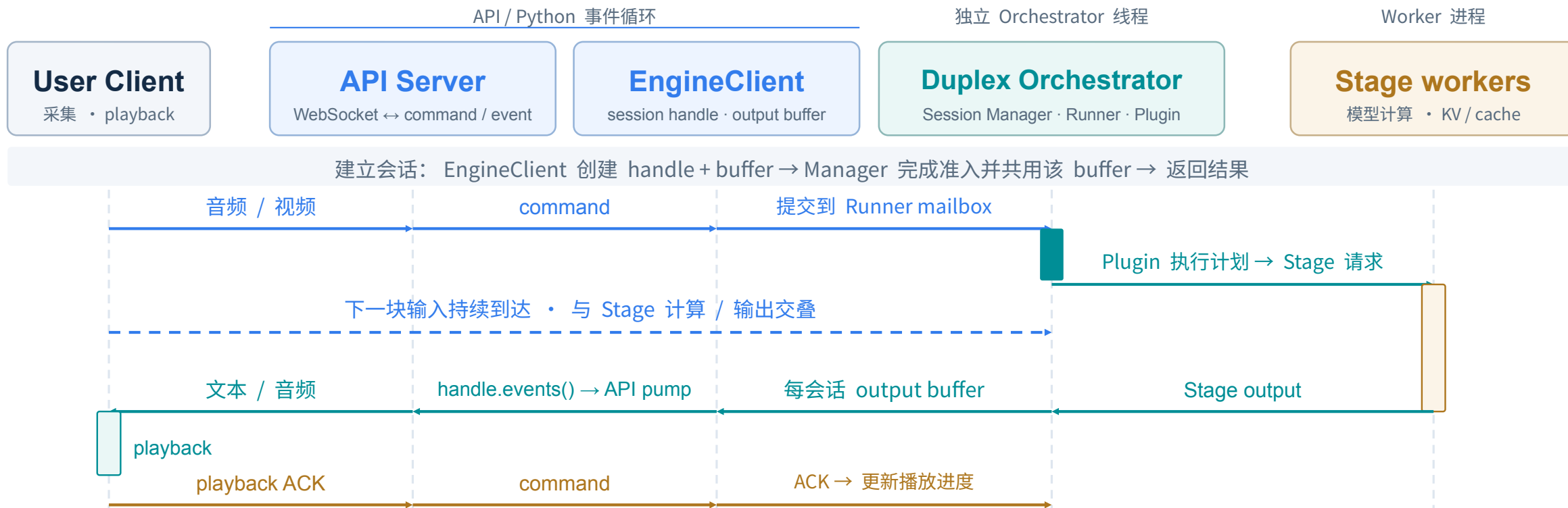
统一全双工架构，通信、会话控制与模型计算按层分工

Duplex WebSocket 与 Python 会话共用引擎侧会话控制、模型插件和有界输出传输。



全双工 workflow

API Server 处理协议； EngineClient 提供 session handle ； Runner / Plugin 驱动执行。



持续输入 · 每个会话的输出有界 · 独立上报播放进度

Session event ACK 在 API 层处理； playback ACK 进入 Runner， history 更新依模型能力而定。

状态处理：持续计算状态与会话历史

MiniCPM 持续更新活跃请求；AURA 与 Qwen3-Omni 按 turn 提交新请求。

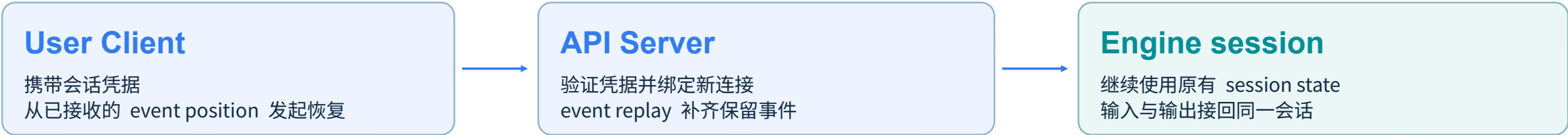
模型	输入处理方式	跨 chunk / turn 保留什么
MiniCPM-o 4.5	追加 chunk ，继续当前请求	LLM KV / 音频状态；可配置 sliding window
AURA	提交 turn ，启动新请求	ASR 文本 / 视频片段 / 助手文本；每轮重建 prompt
Qwen3-Omni	提交 turn ，启动新请求	用户音频 / 图像帧 / 对话文本；每轮重建 prompt



Stage = 一个模型计算环节；Replica = 同一 Stage 的执行实例。
MiniCPM 的 KV / 流式状态续用以对应请求存活为前提；AURA 的 TTS 可在同一 turn 内续算。

恢复连接，延续同一会话

凭会话凭据重新绑定仍存活的 engine session，从上次接收位置续传保留的 event。



依赖同一存活 owner、有效 lease 和保留的 event；session resume 不提供进程故障切换。

恢复环节	框架行为	User Client 接入
会话身份	沿用同一 session	Python SDK 自动维护；自定义客户端保存 session ID 与 resume token
连接绑定	新连接接管原会话	重新建立 WebSocket，发送 session.resume
事件续传	续传保留的 event	携带最后接收的 event position 接收仍在 replay window 内的事件

支持活跃会话续接：MiniCPM · AURA · Qwen3-Omni · Nemotron；PersonaPlex 断连后需新建会话。

优化流式执行，减少重复计算

针对模型优化 CUDA 执行路径，减少实时输入、生成与语音解码中的重复工作。

01 MiniCPM：保留有界 KV window

Basic 模式：按 block 对齐的 sliding window
原地 Re-RoPE，无需复制保留的 KV

02 MiniCPM：音视频 CUDA graphs

流式音频与视觉 encoder 使用 CUDA graphs
Code2Wav 常驻执行，结合 fused kernels

03 PersonaPlex：实时音频帧 batching

跨活跃 session 对追加输入进行 batching
复用 Mimi encoder / decoder CUDA graphs

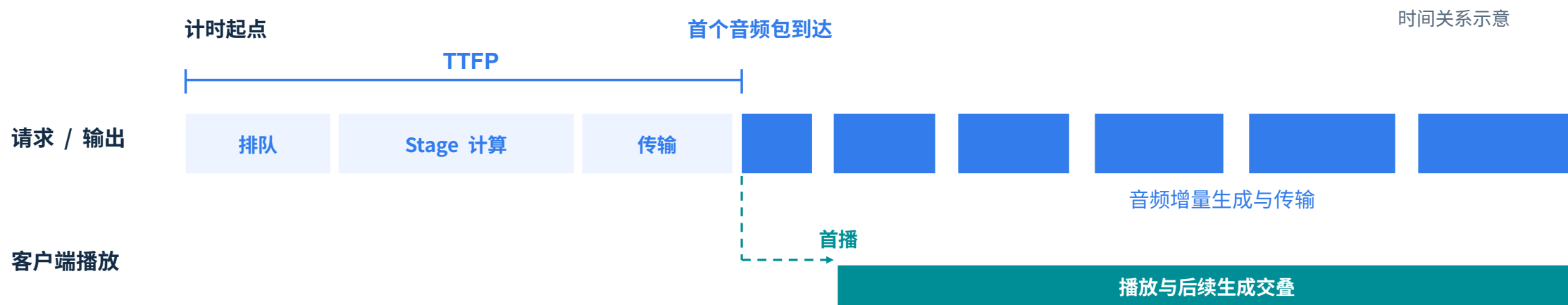
04 Qwen3-Omni：捕获视觉计算图

Thinker image encoder 使用 CUDA graphs
在支持的 shape 下复用已捕获的计算图

选择匹配的 CUDA profile，在目标负载下测量端到端 TTFP 与 RTF。

从首包速度到持续交互质量

OmniInteract 与 Omni-DuplexEval 结合实时会话执行，评估响应时序与交互质量。



响应速度

TTFP • p50 / p95
区分排队、计算与传输

持续输出

RTF • chunk 间隔
停顿与 playback buffer

长会话与并发

持续时长 • session 数
峰值显存 • 资源回收

语音质量

有效音频 • 内容完整性
WER / CER • 听感

统一模型、硬件、并发和计时起点；各模型支持的 benchmark 与测评任务有所不同。

单并发与 4 并发的性能表现

单并发与 4 并发下的响应延迟和语音生成效率。

MiniCPM-o 4.5 • H100 • Seed-TTS 文本转语音

指标（平均值）	单并发	4 并发
TTFT ↓	161.8 ms	252.8 ms
TTFP ↓	635.3 ms	1451.9 ms
RTF ↓	0.27	0.64

AURA • 2×H200 • OmniInteract 实时音视频会话

指标（语音回应中位数）	单并发	4 并发
TTFT ↓	208 ms	244 ms
TTFP ↓	463.8 ms	480.1 ms

TTFT：首个 token 延迟；TTFP：首个音频包延迟；RTF：生成耗时 / 音频时长。均为越低越好。
MiniCPM：2026-09-21 测试记录。AURA：2 FPS，warmup 不计入统计。

全双工模型的工具调用

扩展模型覆盖，统一工具执行期间的交互行为。

当前已支持

Qwen3-Omni Realtime turn-based Realtime 路径已接入 tool parser	Nemotron VoiceChat native duplex 支持工具结果回填与续生成
---	---

Unified duplex 演进方向

模型输出 工具名称 + 参数	→	应用执行 外部函数	→	结果回填 匹配 call_id	→	模型续生成 生成后续回复
执行期间	明确工具执行期间是否继续接收用户输入、输出语音。					
轮次隔离	处理打断、超时、重复及迟到结果，避免旧调用影响新一轮交互。					
框架与插件分工	框架负责公共调用逻辑；插件适配模型输出格式与行为。 扩展至 Qwen3-Omni unified duplex 、 AURA 与 JoyAI 。					

全双工 roadmap

近期完善模型接入、交互能力与测评；远期扩展会话时长和服务规模。

近期

JoyAI 接入

将 JoyAI 接入 unified duplex 框架
完成模型输入与输出适配

通用工具调用

统一调用与结果处理，完善模型适配
处理打断、超时与过期结果

Semantic VAD

结合语义与上下文判断用户是否说完
接入 commit 与 response 策略

扩展实时交互测评

扩展 OmniInteract / Omni-DuplexEval 覆盖
增加模型、交互任务与回归检查

远期

更长且稳定的连续会话

管理长时间音视频会话的 context 与 history
随会话延长，保持稳定的延迟与内存占用

扩大多副本服务规模

通过多个 Stage replica 提升会话并发
支持跨 API frontend 的会话路由与归属管理

全双工执行链路：从输入到模型，再到输出

普通输出通过每个 session 的有界 buffer 传递，由 API 与 Orchestrator 线程共用。



Acoustic VAD 与 Semantic VAD

常见组合：Acoustic VAD 先检测停顿，再由 turn detection 模型判断用户是否说完。

对比维度	Acoustic VAD	Semantic VAD 规划扩展
判断依据	语音活动 + 配置的静音时长	语义与上下文 音频模型还可结合语调等线索
停顿处理	静音达到阈值时可能 commit 即使句子尚未说完	判断话语尚未完整时 可继续等待
系统动作	静音达到阈值后 commit 输入 按配置自动开始 response	判断用户说完后 commit 输入 按配置自动开始 response
计算与部署	CPU 推理开销低 当前：CPU thread pool	单次判断通常需要更多计算 规划：较重模型在 Omni Stage 执行

开源组合示例：SILERO VAD + PIPECAT SMART TURN v3.2

“帮我订明天的…… [停顿] ……去北京的机票。”

1. Silero：区分语音 / 非语音，检测停顿

2. Smart Turn：判断话轮完整 / 尚未完整

VAD 可减少非语音噪声造成的误触发。音频降噪与打断意图识别是独立能力。

VAD：会话层做决策，worker 执行检测

当前 Server VAD 使用 Silero；Semantic VAD 可沿同一职责划分扩展。

当前 Server VAD

Unified duplex：同一 API 进程，控制循环 + CPU 线程



Semantic VAD · 规划扩展



Speech onset 用于及时打断；Semantic VAD 判断用户是否说完，区别于模型 LISTEN / SPEAK。

断连、cancel 与 close ，各自影响什么

MiniCPM 、 AURA 、 Qwen3-Omni 与 Nemotron 支持活跃会话续接； PersonaPlex 断连即关闭会话。

触发操作	正在执行的工作	会话与恢复
断连 · 支持续接的模型	可继续生成；记录后续事件	断连 grace 内、会话仍存活时可续接
response.cancel	取消当前 response ，使其待发音频失效	会话保留；已取消回复不恢复
session.close / lease 过期	终止并清理关联请求	会话结束；重新交互需新建

会话仍存活

在断连 grace 内恢复
session 仍由存活的 owner 持有

会话凭据有效

resume token 有效
lease generation 匹配

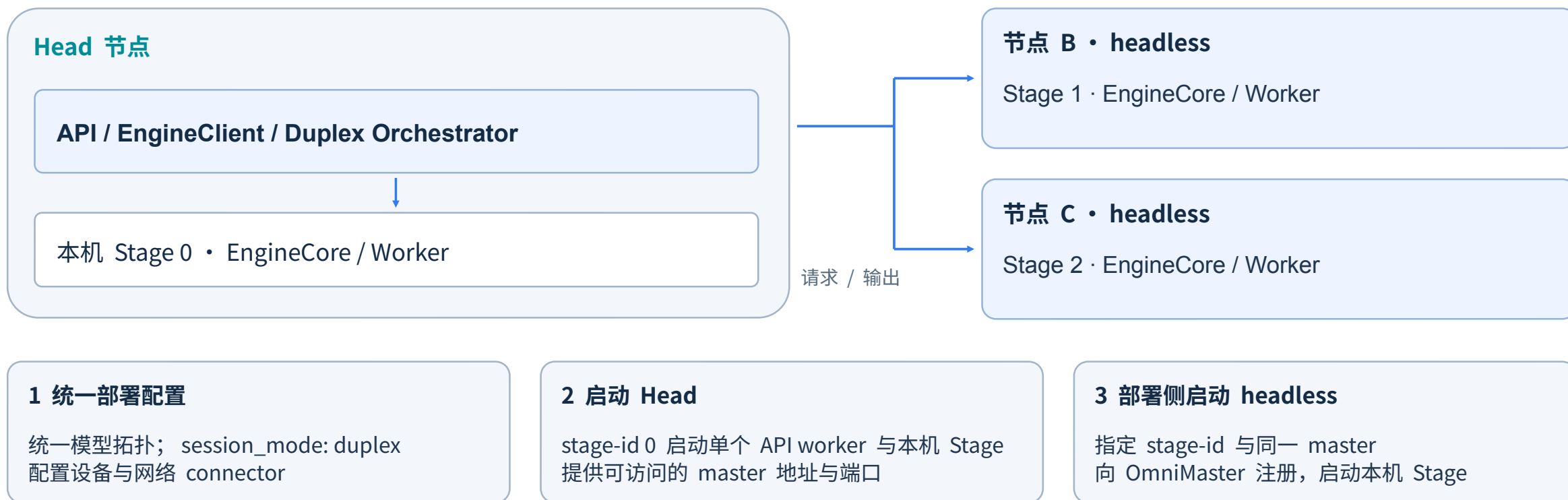
保留事件可续传

提供最后接收的 event position
所需 event 仍在保留范围内

默认：断连 grace 30s · idle TTL 300s · replay 60s / 8 MiB 。恢复依赖同一仍存活的 owner 。

跨机 Stage 共用入口侧的 Orchestrator

单个 API worker 持有 duplex session；Head 连接本机及已注册的 headless Stage 进程。



Headless Stage 共用 Head Orchestrator；duplex 不支持多 API worker，跨机传输需配置网络 connector。

参考代码

层次	实现入口	阅读重点
API 通信适配	OmniDuplexSessionHandler	协议 event、连接绑定与 replay
EngineClient	DuplexOmni / DuplexSessionHandle	handle、共享 output buffer 与 event
通信与传输	DuplexOmniEngine / DuplexOutputBuffer	控制队列与有界 session 输出
会话运行时	DuplexSessionManager / DuplexSessionRunner	lease、状态、mailbox 与模型策略
Stage 编排	DuplexOrchestrator / StagePool	初始请求、后续更新与副本绑定
模型执行	Scheduler / Model Worker	batching、计算与模型缓存